

Final Report

Group 21

Diogo Sousa fc59792
Bruno Faustino fc59784
Rodrigo Neto fc59850

June 14, 2026

Contents

1	Motivation	3
2	Dataset Characterization	4
3	Business Capabilities	4
3.1	Competitive Ranking	4
3.2	Custom Match Management	4
3.3	Custom Match History	4
3.4	Team Draft and Build Analyzer	4
4	Use Cases	4
4.1	Match & Ranking Management	5
4.2	Data Visualization & Analysis	5
5	API Specification	5
5.1	Player Management - Diogo Sousa	5
5.2	Queue Management - Diogo Sousa	6
5.3	Matchmaking - Diogo Sousa	6
5.4	Leaderboard - Diogo Sousa	7
5.5	Player Ranking - Diogo Sousa	7
5.6	Match History - Bruno Faustino	8
5.7	Match Filling - Bruno Faustino	8
5.8	Player Filling - Bruno Faustino	8
5.9	System - Rodrigo Neto	9
5.10	Training - Rodrigo Neto	9
5.11	Datasets - Rodrigo Neto	10
5.12	Games - Rodrigo Neto	10
5.13	Models - Rodrigo Neto	11
5.14	Features - Rodrigo Neto	11
5.15	Analysis - Rodrigo Neto	11
5.16	Authentication — Rodrigo Neto	11
6	Functional Requirements	12
6.1	Matchmaking - Diogo Sousa	12
6.2	Ranking - Diogo Sousa	12
6.3	Match History - Bruno Faustino	12
6.4	Detail Filling - Bruno Faustino	13
6.5	Training - Rodrigo Neto	13

6.6	Data Analysis - Rodrigo Neto	13
7	Non-Functional Requirements	13
7.1	Microservices Architecture - Diogo Sousa	14
7.2	Serverless & FaaS Integration - Bruno Faustino	14
7.3	Data Science & Machine Learning - Rodrigo Neto	14
7.4	Observability Stack - Bruno Faustino	14
7.5	DevOps & Automation - Bruno Faustino & Rodrigo Neto	15
7.6	Security & Identity Management - Rodrigo Neto	15
8	Application Architecture	16
8.1	Matchmaking Service - Diogo Sousa	16
8.2	Ranking Service - Diogo Sousa	16
8.3	Match History Service - Bruno Faustino	16
8.4	Detail Filling Service - Bruno Faustino	16
8.5	Training Service - Rodrigo Neto	17
8.6	Analysis Service - Rodrigo Neto	17
9	Technical Architecture	17
10	Implementation (High Level Description)	18
11	Deployment Scripts	18
11.1	Local Checks	18
11.2	Deployment	18
12	Tuning	19
12.1	Resource Requests and Limits	19
12.2	Scaling Policies	19
12.3	Some Notes about Resource Requests/Limits and Scaling Policies	19
12.4	Health Probes	19
13	CI/CD Pipeline	20
13.1	Continuous Integration (CI)	20
13.2	Continuous Delivery (CD)	20
14	Test, Evaluation Techniques and Results (to target group)	20
14.1	Documentation Feedback - Diogo Sousa	20
14.2	Testing Results - Diogo Sousa	21
14.2.1	Smoke Testing (Endpoint: GET /api/listings/search)	21
14.2.2	Stress Testing (Endpoint: GET /api/listings)	21
14.3	Documentation Feedback - Bruno Faustino	21
14.4	Testing Results - Bruno Faustino	22
14.4.1	Smoke Testing (Endpoint: GET /api/auctions/{auction_id}/bids)	22
14.4.2	Stress Testing (Endpoint: GET /api/market/average_price)	22
14.5	Documentation Feedback - Rodrigo Neto	23
14.6	Testing Results - Rodrigo Neto	23
14.6.1	Smoke Testing (Endpoint: GET /api/market/price-comparison)	23
14.6.2	Stress Testing (Endpoint: GET /api/listings/stats/by-location)	23
15	Corrections Implemented after Feedback from Testing	23
15.1	Improved Error Handling in Player Registration - Diogo Sousa	23
15.2	Standardized Exception Mapping - Diogo Sousa	24

15.3 Documentation Consistency - Diogo Sousa	24
15.4 Clarified Match History Documentation Descriptions - Bruno Faustino	24
15.5 Improved Guidance and Default Definitions - Rodrigo Neto	24
16 Cost Analysis	24
16.1 FinOps and Optimization Measures	25
17 Conclusions	25

List of Figures

1 Application Architecture	16
2 Technical Architecture	17

List of Tables

1 Match & Ranking Management Use Cases per Student	5
2 Data Visualization & Analysis Use Cases per Student	5
3 Player Management REST API	6
4 Queue Management REST API	6
5 Matchmaking REST API	7
6 Leaderboard REST API	7
7 Player Ranking REST API	7
8 Match History REST API	8
9 Match Filling REST API	8
10 Player Filling REST API	8
11 System REST API	9
12 Training REST API	9
13 Datasets REST API	10
14 Games REST API	10
15 Models REST API	11
16 Features REST API	11
17 Analysis REST API	11
18 Authentication REST API	12
19 Estimated Monthly Infrastructure Cost	25

1 Motivation

ScrimFinder was born from the need for a more structured and data-driven approach to competitive "scrims" (practice matches) in League of Legends. While the official game offers matchmaking, it lacks the depth required for teams and individual players to organize balanced practice sessions, track progress through custom-built ranking systems (MMR), and gain actionable insights through post-game ML-driven analysis. By leveraging a massive 78GB dataset of EUW matches, ScrimFinder provides a platform that not only matches players based on skill but also analyzes their performance to offer personalized coaching tips. Our goal was to build a truly cloud-native application that demonstrates high availability, scalability, and advanced data processing capabilities.

2 Dataset Characterization

- **Topic:** League of Legends Matches
 - **URL:** Kaggle - Matches Raw EUW
 - **Size:** Approximately 78 GB
 - **Release Date:** January 2026
-

3 Business Capabilities

3.1 Competitive Ranking

Facilitates real-time visualization of global and local leaderboards. Includes simplified performance metrics such as Damage dealt and KDA ratios.

3.2 Custom Match Management

Enables the creation of balanced custom matches. Matchmaking is handled via a global/local queue system integrated with the project's ranking logic.

3.3 Custom Match History

Provides a centralized repository for past custom match statistics. Includes robust functionality for custom filtering and sorting based on specific user requirements.

3.4 Team Draft and Build Analyzer

Offers personalized post-game analysis. Players utilizing the API receive automated insights regarding gameplay performance and actionable tips for improvement.

4 Use Cases

4.1 Match & Ranking Management

Use Case	Student Name
Leaderboards	Diogo Sousa
User Ranking	Diogo Sousa
Match Finding	Diogo Sousa
Create / Cancel Match	Diogo Sousa

Table 1: Match & Ranking Management Use Cases per Student

4.2 Data Visualization & Analysis

Use Case	Student Name
Match History	Bruno Faustino
Advanced History Filtering / Sorting / Paging	Bruno Faustino
Automatic Detail Filling (by interacting with RIOT's API)	Bruno Faustino
Draft Analysis	Rodrigo Neto
Build Analysis	Rodrigo Neto
Player Performance Analysis	Rodrigo Neto

Table 2: Data Visualization & Analysis Use Cases per Student

5 API Specification

5.1 Player Management - Diogo Sousa

Operations for registering players and linking their Discord accounts with external League of Legends identities.

Verb	Endpoint	Description
POST	/api/v1/matchmaking/ players	Register a new player in the platform.
GET	/api/v1/matchmaking/ players/{id}	Get internal player details.
GET	/api/v1/matchmaking/ players/{id}/lobbies	Get all lobbies associated with a player.
GET	/api/v1/matchmaking/ players/{id}/tickets	Get all active tickets for a player.
DELETE	/api/v1/ranking/players/ links/{gameName}/ {tagLine}	Unlink a League of Legends account.
GET	/api/v1/ranking/players/ {playerId}	Get player profile and MMR.
PUT	/api/v1/ranking/ players/{playerId}/ link-lol-account	Link a Riot account to a player.
GET	/api/v1/ranking/ players/{playerId}/ primary-account	Get the primary linked Riot account.
PUT	/api/v1/ranking/ players/{playerId}/ set-primary-account	Set the primary Riot account.
POST	/api/v1/ranking/players/ {playerId}/sync-mm	Sync MMR from external services.

Table 3: Player Management REST API

5.2 Queue Management - Diogo Sousa

Operations for managing matchmaking queues, defining rules, and setting MMR requirements.

Verb	Endpoint	Description
POST	/api/v1/matchmaking/ queues	Create a new global or local queue.
GET	/api/v1/matchmaking/ queues/{id}	Get queue configuration and status.
PUT	/api/v1/ranking/queue/ {queueId}	Update queue MMR rules.
GET	/api/v1/ranking/queue/ {queueId}	Get queue details from Ranking.
DELETE	/api/v1/ranking/queue/ {queueId}	Delete a queue.

Table 4: Queue Management REST API

5.3 Matchmaking - Diogo Sousa

Core logic for joining queues, accepting match proposals, and completing matches.

Verb	Endpoint	Description
POST	/api/v1/matchmaking/tickets	Join a matchmaking queue.
POST	/api/v1/matchmaking/tickets/matches/{matchId}/complete	Finalize match and report results.
PUT	/api/v1/matchmaking/tickets/matches/{matchId}/link	Link external Game ID to match.
POST	/api/v1/matchmaking/tickets/matches/{matchId}/players/{playerId}/accept	Accept a match proposal.
POST	/api/v1/matchmaking/tickets/matches/{matchId}/players/{playerId}/decline	Decline a match proposal.
DELETE	/api/v1/matchmaking/tickets/{ticketId}	Leave a matchmaking queue.
GET	/api/v1/matchmaking/tickets/{ticketId}/lobby	Get current lobby for a ticket.

Table 5: Matchmaking REST API

5.4 Leaderboard - Diogo Sousa

Retrieval of global and queue-specific rankings.

Verb	Endpoint	Description
GET	/api/v1/ranking/leaderboards	Get global leaderboard (paginated).
GET	/api/v1/ranking/leaderboards/queues/{queueId}	Get queue-specific leaderboard.

Table 6: Leaderboard REST API

5.5 Player Ranking - Diogo Sousa

Operations to track individual player MMR and ranking history across different queues.

Verb	Endpoint	Description
GET	/api/v1/ranking/players/{playerId}/queue-rankings	Get all queue rankings for a player.
GET	/api/v1/ranking/players/{playerId}/queue-rankings/{queueId}	Get ranking for a specific queue.

Table 7: Player Ranking REST API

5.6 Match History - Bruno Faustino

Historical repository of match statistics with advanced filtering, sorting and paging.

Verb	Endpoint	Description
GET	/api/v1/history/matches	Get paginated match history with filters and sorting.
GET	/api/v1/history/matches/{riotMatchId}	Get simplified match information, useful for players.

Table 8: Match History REST API

5.7 Match Filling - Bruno Faustino

Interaction with Riot's API to enrich match data.

Verb	Endpoint	Description
GET	/api/v1/riot/matches/{matchId}	Get simplified stats from Riot API.
GET	/api/v1/riot/matches/{matchId}/raw	Get raw match data for analysis and training.

Table 9: Match Filling REST API

5.8 Player Filling - Bruno Faustino

Retrieval of external player accounts and stats from Riot's API.

Verb	Endpoint	Description
GET	/api/v1/riot/players/{server}/{name}/{tag}	Get Riot account details by summoner name.

Table 10: Player Filling REST API

5.9 System - Rodrigo Neto

Health checks and monitoring endpoints for analysis and training services.

Verb	Endpoint	Description
GET	/api/v1/analysis/	Service root.
GET	/api/v1/analysis/health	General health status.
GET	/api/v1/analysis/q/ health/ready	Readiness probe.
GET	/api/v1/analysis/q/ health/live	Liveness probe.
GET	/api/v1/training/	Service root.
GET	/api/v1/training/health	General health status.
GET	/api/v1/training/q/ health/ready	Readiness probe.
GET	/api/v1/training/q/ health/live	Liveness probe.
GET	/api/v1/auth/health	Service health status

Table 11: System REST API

5.10 Training - Rodrigo Neto

Management of ML training jobs for performance and draft models.

Verb	Endpoint	Description
POST	/api/v1/training/jobs	Start a new ML training job.
GET	/api/v1/training/jobs	List all training jobs.
GET	/api/v1/training/jobs/ {job_id}	Get job status and metrics.
POST	/api/v1/training/jobs/ {job_id}/cancel	Stop a running job.
POST	/api/v1/training/jobs/ {job_id}/deploy	Deploy the trained model.

Table 12: Training REST API

5.11 Datasets - Rodrigo Neto

Creation and curation of training datasets from ingested games.

Verb	Endpoint	Description
POST	/api/v1/training/datasets	Register a new dataset definition.
GET	/api/v1/training/datasets	List curated datasets.
POST	/api/v1/training/datasets/build	Create and build a dataset from games.
GET	/api/v1/training/datasets/{dataset_id}	Get dataset metadata.
DELETE	/api/v1/training/datasets/{dataset_id}	Delete a dataset.
POST	/api/v1/training/datasets/{dataset_id}/build	Rebuild an existing dataset.

Table 13: Datasets REST API

5.12 Games - Rodrigo Neto

Ingestion and retrieval of match data for training purposes.

Verb	Endpoint	Description
POST	/api/v1/training/games	Ingest a single match.
GET	/api/v1/training/games	List all ingested games.
POST	/api/v1/training/games/batch	Ingest matches in batches (up to 1000).
GET	/api/v1/training/games/{game_id}	Fetch stored match details.
DELETE	/api/v1/training/games/{game_id}	Delete an ingested game.

Table 14: Games REST API

5.13 Models - Rodrigo Neto

Registry for versioned ML models and their active status.

Verb	Endpoint	Description
GET	/api/v1/training/models	List all registered models.
GET	/api/v1/training/models/active	List currently active models.
GET	/api/v1/training/models/{model_id}	Get model performance metrics.
DELETE	/api/v1/training/models/{model_id}	Remove a model from registry.
POST	/api/v1/training/models/{model_id}/activate	Set model as active for analysis.
POST	/api/v1/training/models/{model_id}/deactivate	Deactivate a model.

Table 15: Models REST API

5.14 Features - Rodrigo Neto

Extraction of structured feature vectors from raw match data.

Verb	Endpoint	Description
POST	/api/v1/training/features/extract	Extract features from match data.
GET	/api/v1/training/features/{game_id}	Get cached feature vectors.
DELETE	/api/v1/training/features/{game_id}	Delete cached features.

Table 16: Features REST API

5.15 Analysis - Rodrigo Neto

Predictive analysis for drafts, builds, and player performance.

Verb	Endpoint	Description
POST	/api/v1/analysis/draft	Predict team win probability.
POST	/api/v1/analysis/build	Score item build effectiveness.
POST	/api/v1/analysis/player	Aggregated performance benchmarking.
POST	/api/v1/analysis/game	Detailed post-game breakdown.
POST	/api/v1/analysis/champion	Global champion stats from EUW dataset.

Table 17: Analysis REST API

5.16 Authentication — Rodrigo Neto

Secure identity management with RS256-signed JWTs, refresh token rotation, and Istio-compatible JWKS endpoint.

Verb	Endpoint	Description
POST	/api/v1/auth/register	Create account with password validation
POST	/api/v1/auth/login	Authenticate and receive token pair
GET	/api/v1/auth/validate	Validate access token (HTTPBearer)
POST	/api/v1/auth/refresh	Rotate token pair (refresh token)
POST	/api/v1/auth/logout	Invalidate current session
POST	/api/v1/auth/logout-all	Invalidate all user sessions
GET	/api/v1/auth/jwks.json	JWKS for RS256 key distribution

Table 18: Authentication REST API

6 Functional Requirements

6.1 Matchmaking - Diogo Sousa

- Management of the matchmaking queue lifecycle, including ticket creation, status tracking, and cancellation;
- "On Join" matchmaking logic to evaluate and form lobbies immediately upon player entry;
- Implementation of a Standard Queue for 10-player lobby formation;
- Implementation of a Role Queue ensuring team balance across all five roles (TOP, JUNGLE, MID, BOT, SUPP);
- Lobby lifecycle control, including match acceptance/rejection and automatic lobby dissolution on failure;
- Coordination of match states from initial discovery to finalization and result reporting.

6.2 Ranking - Diogo Sousa

- Real-time calculation and updates of player skill ratings (ELO/MMR) based on match performance;
- Management of global and queue-specific leaderboards with paginated retrieval;
- Automatic player profile initialization and rank mapping via Riot account integration;
- Support for multiple ranking systems tailored to specific queues.

6.3 Match History - Bruno Faustino

- Local store of simplified match information relevant to the user (winning team, patch, creation date and length, roles and champions of each player, number of kills, deaths, assists, healing, damage,..., per match, per team and per player), that can be inserted internally by the matchmaking service by providing the match's ID in Riot's API, the id of the queue and the MMR deltas of each player in that match;
- Match filtering based on any criteria the user defines from the simplified information, including obtaining all matches of a given queue, time, patch, player, all matches where

the player had the most kills, played with one of the provided champions, all matches where a team had at most a certain amount of deaths, healing ...;

- Match sorting based on match queue, time, patch and team stats;
- Match retrieval with paging to avoid fetching enormous amounts of data all at once;
- Soft deletion of matches from the history;
- New match data forwarding to the data analysers for training.

6.4 Detail Filling - Bruno Faustino

- Direct contact with Riot's API to obtain complete match and player information;
- Automatic filtering of information to focus more on the user's needs;
- Retrieval of the full raw data to be used for data training and analysis;
- Retrieval of player information (account details, region, rank, wins, losses, points, ...).

6.5 Training - Rodrigo Neto

- Ingestion of raw match data in batches with automatic deduplication and metadata storage;
- Extraction and versioning of structured feature vectors for different analytical models;
- Creation and management of curated, optimized training datasets with configurable filtering;
- Asynchronous execution of ML training jobs for draft, build, and performance models;
- Real-time monitoring of training progress and lifecycle management of active jobs;
- Maintenance of a versioned model registry with performance metrics.

6.6 Data Analysis - Rodrigo Neto

- Win probability prediction for team compositions;
 - Identification of team synergies, counter-matchups, and win conditions;
 - Evaluation of item build effectiveness and possibly suggestion of alternatives;
 - Automated benchmarking of player performance metrics against global/rank-tier averages;
 - Global champion performance tracking including win, pick, and ban rate analytics.
-

7 Non-Functional Requirements

In accordance with the project guidelines, we implemented several non-functional improvements to ensure the system is robust, scalable, and secure.

7.1 Microservices Architecture - Diogo Sousa

The core architecture focuses on high availability and loose coupling through message-driven patterns.

- **API Gateway:** Utilization of **Traefik** for client request routing, rate limiting and API version management due to its native integration with Docker and Kubernetes, and ease of configuration.
- **Database per Service:** Each microservice (Matchmaking, Ranking, History) maintains its own isolated PostgreSQL database, ensuring data encapsulation and independent scaling. Critical databases utilize a **Master-Slave replication model** to ensure high availability and read-scalability.
- **Retries** - Everyone: Failed API requests are resent a configured amount of times before being officially reported as a failure to prevent occasional communication errors from degrading the system.
- **Timeouts** - Everyone: API requests taking too long to provide an answer are stopped early to contribute to the **fail-fast** pattern.
- **Circuit Breaker** - Everyone: After a configured number of API request failures to a service, posterior request attempts to that service will immediately fail for some time instead of taking long to warn the answer of what will likely be another failure, contributing to the **fail-fast** pattern.
- **Cache Aside** (Performance, Availability) - Bruno Faustino: Utilization of a **Redis** cache to store Riot's API responses to reduce the number of external API calls.
- **Message-Driven (Pub/Sub):** Utilization of **RabbitMQ** for asynchronous, decoupled communication (e.g., Matchmaking publishing game results that are consumed by History and Training services).

7.2 Serverless & FaaS Integration - Bruno Faustino

To optimize resource usage and maintenance, specific background tasks (such as the 3 Detail Filling endpoints via Riot's API) were designed to transition to a Serverless model.

- **Implementation:** Quarkus' **Google Cloud Functions** extension for a function-as-a-service (FaaS) experience to handle bursty API polling, integrated with the cloud provider of this project.

7.3 Data Science & Machine Learning - Rodrigo Neto

- **Batch ML Pipelines:** Engineered a dedicated Training service using **scikit-learn** and **LightGBM** to process the 78GB dataset in batches, extracting features and training predictive models without impacting real-time API performance.

7.4 Observability Stack - Bruno Faustino

¹ Utilization of **Grafana Cloud** and its integrated tools (**LGTM**) for monitoring metrics, logs and traces:

- **Loki:** Log aggregation system using label-based indexing for cost-efficiency.

¹This requirement was not complete at the time of writing so its monthly cost will not be considered in the FinOps section.

- **Grafana:** Central visualization hub for dashboards and alerting.
- **Tempo:** High-scale distributed tracing backend.
- **Mimir/Prometheus:** Long-term storage and collection of time-series metrics.
- **Alloy:** OpenTelemetry-native collector for forwarding metrics, logs and traces data.

7.5 DevOps & Automation - Bruno Faustino & Rodrigo Neto

- **Infrastructure as Code (IaC)** - Rodrigo Neto: Provisioning Google Kubernetes Engine (GKE) and secrets management using **Terraform**.
- **Configuration Management**¹ - Bruno Faustino: **Ansible** to manage Kubernetes ConfigMaps and software state.
- **GitOps** - Bruno Faustino: **Argo CD** is used to automatically synchronize the Kubernetes cluster state with the Git repository.

7.6 Security & Identity Management - Rodrigo Neto

¹Security is enforced at both the application level (**JWT**) and the network level (**Service Mesh**).

- **Authentication Microservice (AMS):** Manages the user lifecycle:
 1. **Account Creation:** AMS validates credentials and stores the user hash in a database.
 2. **Authentication:** User logs in, AMS validates credentials, generates a **JWT**, and stores it in a **Redis** cache for session management.
 3. **Request Validation:** The user provides the JWT in request headers. Microservices validate the token with AMS (or via Service Mesh policy) to grant access.
 - **Istio Service Mesh:** Enforces **mTLS** (mutual TLS) for secure service-to-service communication and decentralized authorization.
 - **Discord Bot Validation:** Action validation between the Discord bot and backend services is secured via a shared secret.
 - Requests are encrypted with the shared secret, including an account ID and timestamp to prevent replay attacks.
-

8 Application Architecture

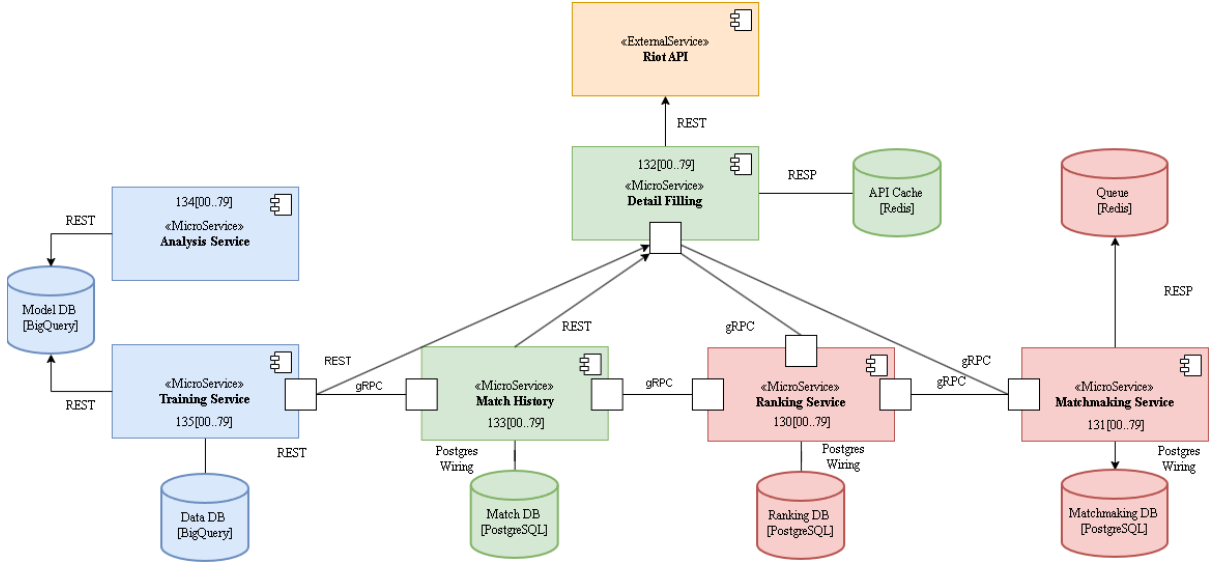


Figure 1: Application Architecture

8.1 Matchmaking Service - Diogo Sousa

- **Database: Redis (Queue/Locks) + PostgreSQL (Lobby Persistence)**
- **Usage:**
 - **Redis:** Manages the active matchmaking tickets and distributed locks for thread-safe lobby formation.
 - **PostgreSQL:** Stores Lobby, Match, and MatchTicket entities to track the lifecycle of a match. Utilizes **Master-Slave replication** for durability.

8.2 Ranking Service - Diogo Sousa

- **Database: PostgreSQL (Ranking DB)**
- **Usage:** Persists long-term player profiles, Discord/Riot account linkages, and MMR history. Configured with **Master-Slave replication** for high availability.

8.3 Match History Service - Bruno Faustino

- **Database: PostgreSQL (History DB)**
- **Usage:** Indexes simplified match statistics for players and teams, supporting complex filtering, sorting and paging. Uses **Read-Replicas (Slave nodes)** to offload heavy filtering queries.

8.4 Detail Filling Service - Bruno Faustino

- **Database: Redis (API Cache)**
- **Usage:** Caches Riot API responses for match and player details to minimize external API calls.

8.5 Training Service - Rodrigo Neto

- **Database: BigQuery (Model DB) / Persistent Volume**
- **Usage:** Responsible for training and storing machine learning models using historical match data.

8.6 Analysis Service - Rodrigo Neto

- **Usage:** Consumes models to provide real-time and post-game insights (draft analysis, performance metrics).

9 Technical Architecture

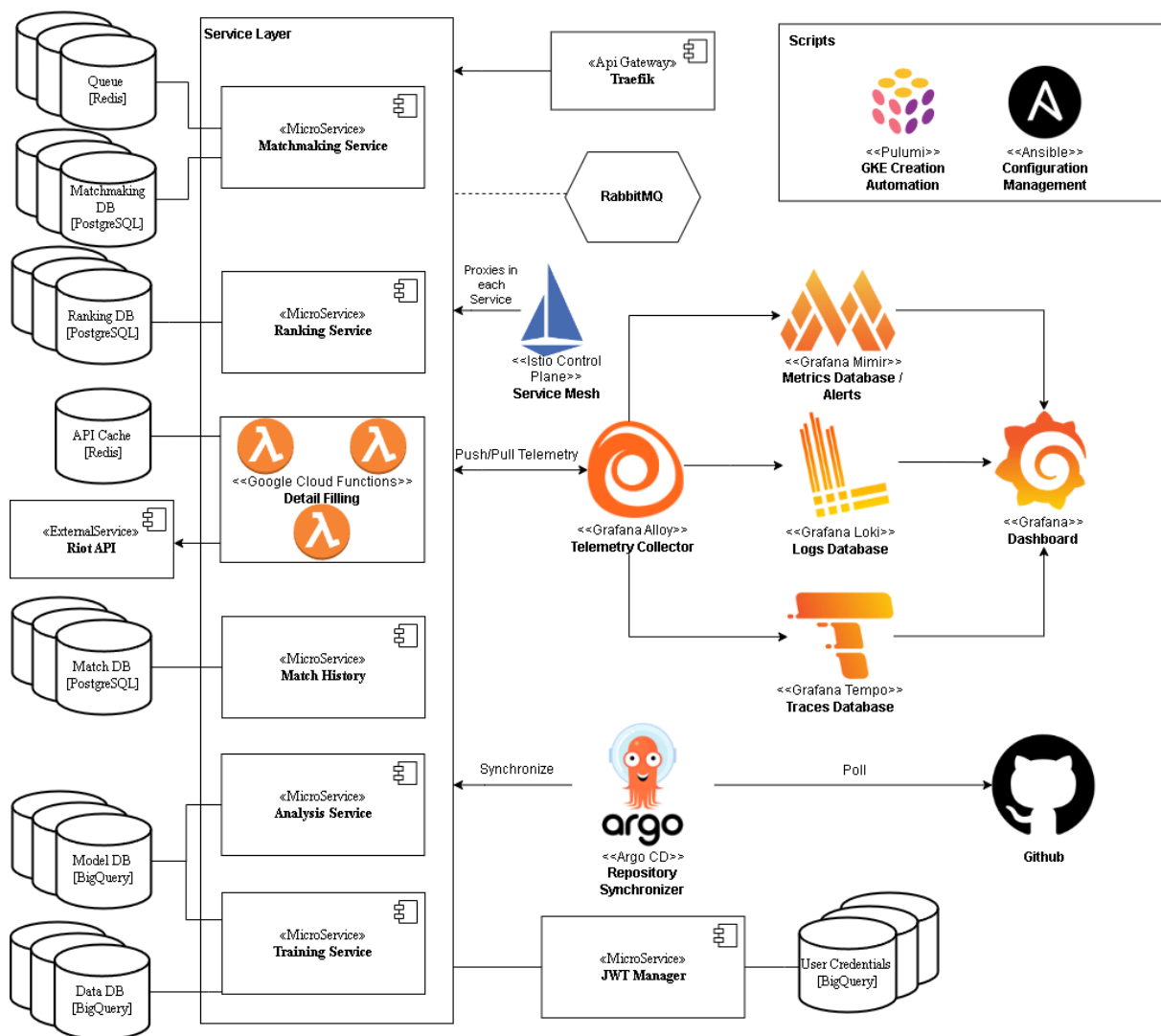


Figure 2: Technical Architecture

10 Implementation (High Level Description)

The system is implemented as a suite of polyglot microservices:

- **Core Backend (Java/Quarkus):** The Matchmaking, Ranking, History, and Detail Filling services are built using Java 21 and the Quarkus framework. This choice provides excellent performance, low memory footprint (ideal for Kubernetes), and native support for gRPC, REST (RESTEasy), and Hibernate ORM for PostgreSQL interaction.
 - **Data Science Backend (Python/FastAPI):** The Training and Analysis services are built in Python using FastAPI. This allows us to leverage the extensive Python ML ecosystem (Pandas, scikit-learn, LightGBM) while maintaining high-performance asynchronous API endpoints.
 - **Communication:** Synchronous inter-service communication is handled via strict **gRPC** contracts (compiled for Java and Python), while asynchronous events (like match completions) are broadcasted via **RabbitMQ** exchanges.
-

11 Deployment Scripts

11.1 Local Checks

```
./scripts/format.sh
./scripts/test.sh
```

Python gRPC files are generated, not committed:

```
./scripts/python/generate-grpc.sh
```

11.2 Deployment

Required environment:

```
export SCRIM_PROJECT_ID="your-gcp-project "
export SCRIM_REGION="europe-west1 "
export SCRIM_REPO_NAME="scrimfinder "
export SCRIM_CLUSTER_NAME="scrimfinder "
export RIOT_API_KEY="RGAPI-..."
export SCRIM_DB_USER="scrim_admin "
export SCRIM_DB_PASSWORD="secure_password "
```

Deploy infrastructure, images, Cloud Functions, and the Helm application to a Google Cloud GKE cluster:

```
./scripts/deploy.sh
```

Tear down the stack:

```
./scripts/shutdown.sh
```

12 Tuning

System performance is optimized through precise resource allocation and automatic scaling policies.

12.1 Resource Requests and Limits

Each service is configured with baseline requests and hard limits to ensure fair scheduling and prevent OOM (Out of Memory) events.

- **Java Services:** Typically 400m-500m CPU and 512Mi Memory.
- **Training Service:** High resource allocation (600m CPU, 2Gi Memory) to handle ML workloads.
- **Analysis Service:** 500m CPU and 1Gi Memory for real-time inference.

12.2 Scaling Policies

- **Horizontal Pod Autoscaler (HPA):** Enabled for all services, scaling from 1 to 3 replicas based on a 70-80% **CPU** utilization threshold.
- **Vertical Pod Autoscaler (VPA):** Enabled for all services, automatically adjusting the **memory** needs of each pod (restarting them if needed), transpassing Argo CD cluster sync. It will keep the ratio between memory requests and limits, which is 1.0.

12.3 Some Notes about Resource Requests/Limits and Scaling Policies

- **CPU Requests:** Set for the HPA.
- **CPU Limits:** Unset to prevent pods from becoming slower not due to actual CPU pressure but due to an unnecessary limit we would be imposing (CPU is compressible so the pod's CPU would be throttled instead of the pod being killed).
- **Memory:** The requests are the same as the limits so that the pod falls on Kubernetes' **Guaranteed QoS class** for memory, meaning the pod will be one of the last to be evicted during memory pressure, while the VPA will adjust both accordingly.
- **HPA and VPA Together:** While the cluster has both, they each target a different resource type (HPA → CPU, VPA → Memory), so they will not conflict with each other.

12.4 Health Probes

Comprehensive health monitoring is implemented using SmallRye Health:

- **Readiness Probes:** Ensure traffic is only routed if the pod is answering to the health requests. Called more often than the liveness probe to stop sending traffic for some time before officially restarting the pod if it still hasn't answered.

- **Liveness Probes:** Automatically restart pods that enter a deadlocked state.
 - **Startup Probes:** Configured with extended failure thresholds (up to 60 for Training service) to accommodate longer initialization times during cold starts.
-

13 CI/CD Pipeline

The ScrimFinder automation pipeline is implemented via GitHub Actions, providing an exhaustive validation flow for every contribution.

13.1 Continuous Integration (CI)

- **Fast Tests:** On every push to `main` or `work/**`, a matrix job runs unit and light integration tests for all Java (Quarkus) and Python (FastAPI) services. This includes linting with `spotless` and `ruff`.
- **Heavy Tests:** Pull requests trigger a complete integration suite.
- **Ephemeral Stacks:** For critical validation, the CI provisions a temporary GKE cluster for staging using Terraform, deploys the entire application stack via Helm/Argo CD, and runs system-wide tests against the live environment before tearing it down.

13.2 Continuous Delivery (CD)

- **GitOps with Argo CD:** The production state is managed in the `k8s/application.yaml` manifest. Argo CD monitors the main branch of the Git repository and automatically synchronizes the cluster state whenever an admin asks Argo CD to synchronize it with the production GKE cluster.
-

14 Test, Evaluation Techniques and Results (to target group)

14.1 Documentation Feedback - Diogo Sousa

- **Style Discrepancy:** The query parameter for pagination size is documented as `pageSize` (camelCase) while the response object uses `page_size` (snake_case). The API Gateway correctly accepts the `pageSize` parameter and returns the corresponding schema, but this inconsistency should ideally be unified.
- **Error Response Documentation:** While standard HTTP error codes (400, 401, 404, 500, 502) are consistently mapped across endpoints using reusable components (e.g., `BadRequestText`), they could be more descriptive. The schema indicates they return a `text/plain` string without explaining *what* triggers them (e.g., what specific validation failure causes a 400, or under what conditions a 404 is thrown for a specific endpoint).

14.2 Testing Results - Diogo Sousa

14.2.1 Smoke Testing (Endpoint: [GET/api/listings/search](#))

The smoke test was executed using a **Python script** checking the endpoint. The test validated the HTTP 200 response, JSON validity, and the presence of all required top-level fields (`total`, `page`, `page_size`, `listings`) and listing level attributes.

- **Basic Paginated Search:** Passed. Returned 5 listings with a reported total of 50,000 records. Latency: $\approx 141\text{ms}$.
- **Filtered Search (Toyota Corolla):** Passed. Returned 5 listings with a reported total of 300 records. Latency: $\approx 191\text{ms}$.

No broken links or malfunctioning behaviors were detected.

14.2.2 Stress Testing (Endpoint: [GET/api/listings](#))

The stress test was executed using **Locust** against endpoint.

- **Concurrency Settings:** 100 concurrent users, spawning at a rate of 20 users per second, run duration of 30 seconds.
- **Performance observed:**
 - **Total Requests:** 6,899
 - **Failures:** 0 (0.00%)
 - **Requests Per Second (RPS):** Peaked at ≈ 275 req/s (average 248.81 req/s over the duration).
 - **Latency Statistics:** Average = 72ms, Median = 62ms, 95th percentile = 93ms, 99th percentile = 150ms.

The endpoint proved to be highly stable under the tested load.

14.3 Documentation Feedback - Bruno Faustino

- **Some positives:**
 - The API documentation was in general easy to understand due to the modularity on the groups of endpoints and the descriptive information tagged with each endpoint.
 - The introductory paragraph gives a quick overview of all the use cases of the application and in which modules they can be found.
 - In general, the exceptions portray in which cases the app may fail (although they could be explained in more detail according to the endpoint in some cases).
- **A few suggestions:**
 - For the chat endpoints, a 404 code should be thrown if the chat id does not exist in the database.
 - It would be helpful to have default values already filled-in in the body of the POST methods that serve as an example to further improve the understandability of the API (for some fields it is not trivial what the expected format is, like `fuel_type`, `drive_type` and `end_time`).

- Although it is well explained that the user needs to add the Host header before calling the post/put/delete endpoints, it is never explained that the user needs to be logged in and that the resulting token needs to be added to the header: a quick mention in either the introductory paragraph or the 401 Unauthorized exception in the documentation would generate less confusion and save a few 401s.

14.4 Testing Results - Bruno Faustino

14.4.1 Smoke Testing (Endpoint: **GET**/api/auctions/{auction_id}/bids)

The smoke test was executed using **pytest** and **requests** modules of **Python**.

Here is the sequence of API calls as a setup before testing the actual endpoint:

1. **POST**/api/user/login for the auth token.
2. **POST**/api/listings for listing_id.
3. **POST**/api/auctions for auction_id.
4. 3x **POST**/api/auctions/{auction_id}/bid.

After calling the endpoint: **GET**/api/auctions/{auction_id}/bids, it is expected that the command returns successfully (200) and with 3 auction bids in the created auction for the created listing.

The smoke test passed!

14.4.2 Stress Testing (Endpoint: **GET**/api/market/average_price)

The stress test was executed using **Locust** against the endpoint.

- **Concurrency Settings:** 10000 concurrent users, spawning at a rate of 5 users per second, running until failing.
- **Performance observed:**
 - **Total Requests:** 250
 - **Failures:** 4 (1.60%)
 - **Requests Per Second (RPS):** Very inconsistent, around 19.61 req/s.
 - **Latency Statistics:** Average = 1009ms, Median = 190ms, 95th percentile = 4000ms, 99th percentile = 6700ms.

The system fails a few times at 19.61 req/s due to the unavailability of an upstream service (502).

During other rounds of testing, the system could fail at less or more than that value, sometimes managing to survive without failures until around 50 req/s, so it is inconsistent, and the number of failures oscillates a lot between load tests, up to the point of reaching 99% failures.

The endpoint proved unable to sustain loads above 50 req/s, and shows signs of failing around 19.61 req/s.

14.5 Documentation Feedback - Rodrigo Neto

- **Missing Content-Type Header:** Successful JSON responses do not include the `Content-Type: application/json` header, which can break strict HTTP clients.
- **Poor Overload Behaviour:** Under high load, the backend does not gracefully degrade (e.g., return 429 or 503). Instead, the gateway returns 502 `Bad Gateway` after extremely long timeouts (up to 47 seconds).

14.6 Testing Results - Rodrigo Neto

14.6.1 Smoke Testing (Endpoint: `GET /api/market/price-comparison`)

The smoke test was executed using **Postman** checking the endpoint. The test validated HTTP 200 responses, JSON validity, pagination, grouping, and sorting behaviors.

- **Happy Path:** Passed with warnings. Valid JSON, correct pagination and sorting. `Content-Type` assertion failed (header omitted).
- **Validation:** Partial pass (3/6). Missing parameters returned 400. Invalid enum values returned 502 `Bad Gateway` instead of 400.

Some malfunctioning behaviors were detected during validation tests.

14.6.2 Stress Testing (Endpoint: `GET /api/listings/stats/by-location`)

The stress test was executed using **Locust** against the endpoint.

- **Concurrency Settings:** Step-load ramping from 10 to 200 concurrent users.
- **Performance observed:**
 - **Total Requests:** 3,524
 - **Failures:** 123 (3.49%)
 - **Requests Per Second (RPS):** Peaked at ≈ 50 req/s.
 - **Latency Statistics:** Average = 7,280ms, Median = 460ms, 95th percentile = 34,000ms, 99th percentile = 38,000ms.

The endpoint proved unable to sustain loads above ≈ 50 RPS, leaving the upstream backend in an unhealthy state.

15 Corrections Implemented after Feedback from Testing

Following the **Cloudy Day** testing phase and internal smoke tests, several critical improvements were implemented to enhance system reliability and client experience.

15.1 Improved Error Handling in Player Registration - Diogo Sousa

Initial feedback indicated that the `POST /api/v1/matchmaking/players` endpoint returned a 500 `Internal Server Error` when a duplicate UUID was provided, whereas a duplicate

Discord username correctly returned a 409 `Conflict`. We updated the `PlayerServiceImpl` to perform an explicit check for existing IDs before persistence. This ensures that all conflict scenarios (ID or Username) return a clear 409 status with a specific error message, improving the API's predictability.

15.2 Standardized Exception Mapping - Diogo Sousa

We refined the `GlobalExceptionHandler` to move away from generic `REMOTE_SERVICE_ERROR` codes for client-side issues. HTTP status codes are now accurately reflected in the error response payload, using `BAD_REQUEST` or `NOT_FOUND` where appropriate. This directly addresses feedback regarding the ambiguity of error messages under high load and validation failures.

15.3 Documentation Consistency - Diogo Sousa

OpenAPI annotations were updated to explicitly state uniqueness constraints and provide example payloads for complex enums, reducing the "trial and error" approach previously required by external testers.

15.4 Clarified Match History Documentation Descriptions - Bruno Faustino

OpenAPI match history descriptions were updated to inform users that the matches only show up in the history after being added via the matchmaking service API and to clarify to users that the history filters must be provided in the body of the request and not as request query parameters, due to size limitations of the request endpoint.

15.5 Improved Guidance and Default Definitions - Rodrigo Neto

The instructions and descriptions were insufficient to allow the intended use of certain endpoints, specially the ones whose default fields cause errors if unset. Added better descriptions and default handling.

16 Cost Analysis

The following table outlines the estimated monthly operational costs for ScrimFinder on Google Cloud Platform (Region: `europa-west1`), calculated using the Official GCP Pricing Calculator. In this architecture, all databases (PostgreSQL, Redis) and message brokers (RabbitMQ) are self-hosted directly on the GKE cluster using persistent storage, rather than managed cloud services. Costs are estimated in Euros (€).

Component	Details	Est. Cost (€)
GKE Cluster ²	3-5 nodes (e2-standard-4 Spot) + Management fee	€170.00
Persistent Storage ³	SSD Storage for PostgreSQL, Redis, and AI data	€145.00
BigQuery ⁴	78GB active storage + Analysis slots	€28.00
Other ⁵	Artifact Registry, Cloud Functions, Traffic	€14.00
Total		€357.00

Table 19: Estimated Monthly Infrastructure Cost

16.1 FinOps and Optimization Measures

To keep the platform cost-effective, we implemented the following strategies:

- **BigQuery for ML:** Offloading massive match history storage to BigQuery instead of relational SQL databases significantly reduces storage costs.
- **Redis Caching:** Aggressively caching Riot API responses reduces the need for expensive high-concurrency calls and helps stay within lower API tier limits.
- **Autoscaling:** HPA ensures we only run multiple replicas during peak hours, while VPA helps identify over-provisioned services that can be downsized.

17 Conclusions

ScrimFinder is a cloud-native platform for competitive League of Legends practice matches, built as a multi-language microservices architecture integrating Java/Quarkus services for real-time matchmaking, ranking, and match history with Python/FastAPI services for ML-driven predictive analysis and authentication. The platform processes a 78GB dataset of EUW matches to provide skill-based matchmaking, post-game analytics, and personalized performance insights.

Functionally, the system covers five core domains. The Matchmaking Service manages queue lifecycle, lobby formation, and match state coordination with support for both standard and role-based queues. The Ranking Service handles ELO/MMR calculation, leaderboard management, and Riot account integration. The Match History Service provides a centralized repository with advanced filtering, sorting, and paging capabilities. The Detail Filling Service interfaces directly with Riot’s API to enrich match and player data, caching responses to minimize external calls and support downstream services. The Training and Analysis Services form a complete ML pipeline — from raw match ingestion and feature extraction through dataset curation, model training, and real-time inference for draft win-probability, build scoring, and player benchmarking.

²GKE Pricing: Scaled to handle both application logic and stateful database workloads.

³Persistent Disk Pricing: Based on regional SSD (pd-ssd) for high-IOPS database performance.

⁴BigQuery Pricing: Scaled for the training service’s feature extraction workloads.

⁵Includes external load balancer and public IP addresses.

From a non-functional perspective, the architecture implements several cloud-native resilience patterns. Retries, timeouts, and circuit breakers protect inter-service communication from cascading failures. Redis caching minimizes external API calls to Riot’s services. RabbitMQ enables asynchronous, decoupled event propagation between Matchmaking, History, and Training services. The Detail Filling Service leverages serverless Google Cloud Functions for cost-effective bursty API polling. Infrastructure is provisioned via Terraform, deployed through Helm, and synchronized via Argo CD GitOps workflows. The Authentication Service secures the entire platform with RS256 token issuance, session management, refresh token rotation, and Istio-compatible JWKS endpoint distribution.

Resource tuning reflects deliberate trade-offs between performance and stability. Java services run with 400–500m CPU requests and 512Mi memory, while Python ML services receive higher allocations (600m CPU, 2Gi memory for Training; 500m CPU, 1Gi memory for Analysis) to accommodate model loading and inference workloads. CPU limits are intentionally unset to prevent throttling under legitimate load, while memory requests equal limits to guarantee Kubernetes QoS class. HPA scales replicas from 1 to 3 based on 70–80% CPU utilization, and VPA adjusts memory requirements independently without conflicting. Health probes — readiness, liveness, and startup — ensure traffic is only routed to healthy pods, with extended startup thresholds for ML services with longer cold-start times.

The estimated monthly operational cost of €357 on GKE Spot instances, self-hosted PostgreSQL/Redis/RabbitMQ, and BigQuery storage represents a pragmatic balance between capability and affordability for a student project.

Beyond the technical deliverable, this project was fundamentally an exercise in learning cloud-native architecture through direct experience. Implementing patterns like circuit breakers and health probes in response to actual CI failures and production timeouts, rather than as theoretical exercises, provided a concrete understanding of why these patterns matter. Working across language boundaries with gRPC contracts, managing infrastructure as code with Terraform, and debugging distributed system behavior in ephemeral GKE clusters were all challenges that no classroom assignment could fully replicate. The fact that we went through building, breaking, and rebuilding a system of this complexity, completely from scratch, making use of many tools we did not dream existed, as a team of three students was a different and very demanding episode of our academic journey, that despite the hardships allowed us to overcome troubles and ultimately get a lot of experience that we would have never gotten otherwise. We hope, this adventure will prove to be valuable in the future (but we hope it does not repeat itself).
